

中文法律議題前置分流

從日常法律情境進行多標籤 NLP 分類

課程：自然語言處理 (NLP) · 期末專案 系統名稱：LexTag — 法律議題前置分流 免責聲明：本專案為 NLP 教學用途的法律議題分流，不提供法律意見、不生成判決。

1. 研究問題 (Research Question)

一般人會用日常中文描述法律困擾（「有人冒用我的照片和名字發文罵人」），卻往往不知道自己的處境屬於哪一類法律議題。現有的台灣法律 AI 產品（Lawsnote、Lawbot、判決搜尋）大多從「下一步」開始：它們假設使用者已經知道自己有妨害名譽、契約或詐欺的問題。

本專案聚焦一個問題：

NLP 分類器能否在任何檢索、聊天機器人或法律意見步驟「之前」，把一段自由輸入的中文法律情境對應到一個或多個法律議題標籤，並把相關法條全文當作查詢線索呈現？

交付物是一個前置分流分類器，不是聊天機器人。輸出為一組議題標籤（如 酒駕/公共危險、詐欺、妨害名譽）、信心分數，以及作為參考線索的支持法條全文。

2. 實務應用 (Practical Application)

這是課程評分占比最高的項目（30%），因此主導整個設計方向。

為何重要。法律問答與判決 RAG 已是紅海，且高度依賴大型專有資料庫。議題「分流」則是尚未解決的上游瓶頸：每一個法律扶助專線、律師事務所的初談表單、法律資訊網站，都需要先把當事人的故事導向正確的法律領域，後續才能進行。

誰受益。

- 法律扶助初談服務 — 自動把進線需求導向正確的值班律師 / 類別，降低人工分流負擔。
- 法律資訊網站 — 把自由文字的抱怨轉成正確的 FAQ / 文章分類。
- 律師事務所初談 — 在第一次諮詢前，先預填結構化的議題標籤與候選法條。
- 法律系學生 — 練習把白話事實連結到法律概念與背後的條文。

如何解決實務問題。LexTag 不與完整法律 AI 產品正面競爭，而是切入便宜、可量化、高槓桿的第一步：分類。因為輸出是標籤集合（而非自由文字建議），任務是可量化的（F1、recall、precision）且可稽核的；同時避免了生成法律意見的責任與幻覺風險。法條全文查詢讓使用者得到具體、可驗證的起點，而系統從不宣稱該條文「適用」於其個案。

3. 資料集說明 (Dataset Description)

3.1 來源

資料集	規模	在本專案的角色
lianghsun/tw-legal-synthetic-qa	9,631 組問答	主要弱標來源 — 中文法律情境 (user) + 分析 (assistant)
lianghsun/tw-processed-law-article	230,974 筆法條	法條查詢 + 本體論支援 — 提供 demo 顯示的法條全文

問答資料集為 ShareGPT 訊息格式。我們把 **user** 訊息當作待分類情境，並僅把 **assistant** 訊息當作弱標訊號，因為分析內容通常會引用具體條文（如 **刑法第185條之3**）。

3.2 前處理流程

- 引文正規化 (`scripts/citation_normalizer.py`) — 透過 NFKC 正規化 + 正規表示式 + 中文數字解析，把多樣的中文引用格式（**刑法第185條之3**、阿拉伯 / 中文數字、**臺/台** 變體、全形空白）轉成穩定的（**法規**，**條號**）鍵。
- 本體論建構 (`data/issues.yaml`) — 手工建立的 **20** 個日常法律議題本體論（12 刑事、8 民事），每個議題對應其支持法條、別名與關鍵字。
- 弱標註 (`scripts/build_labels.py`) — 從每筆 assistant 答案抽出引文，經本體論對應為議題標籤；只保留至少 1 個標籤且情境 ≥ 20 字的資料。
- 法條全文查詢 (`scripts/build_law_lookup.py`) — 針對本體論引用的 26 組（**法規**，**條號**），從 `tw-processed-law-article` 抽出完整條文寫入 `data/law_articles.json`（去除「<法規> 第 N 條」標題、保留本文）。26 條全數解析成功（刑事 13、民事 13）。
- 固定切分 (`scripts/split_data.py`) — 以固定亂數種子做 70/15/15 的 train/val/test 切分。

3.3 最終資料集

- **1,523** 筆弱標資料 → 訓練 **1,066** / 驗證 **228** / 測試 **229**。

3.4 限制 (誠實陳述)

- 弱標非黃金標 — 標籤由合成的法律分析推導而來，確實存在標籤雜訊。
- 嚴重類別不平衡 — 光是 **酒駕/公共危險** 就佔 672/1,523 (**44%**)，而最稀有的類別（**所有物返還**、**繼承**、**夫妻財產**）各只有 9-11 筆。
- 領域偏差 — 僅涵蓋刑事與民事實體法；程序法刻意排除（這是日常議題分流，不是法院程序分類）。
- 不保證法律適用性 — 呈現的條文僅為查詢線索，絕不代表它規範該特定個案。

4. NLP 方法 (NLP Method)

整個流程分五個階段，直接對應課程主題（斷詞、TF-IDF、文本分類）：

1. 引文正規化 — 正規表示式 + 中文數字解析 → 穩定的條號鍵。
2. 本體論對應 — 高頻條文 → 20 個議題標籤。
3. 弱標註 — 由引文推導的多標籤目標。
4. 分類 — 兩種對比方法（見下）。
5. 評估與服務 — 多標籤指標 + Flask API + Next.js demo。

方法 **A** — 規則比對基準（`baselines/rule.py`）。當情境出現某議題的任一別名 / 關鍵字即觸發該議題，信心隨命中數上升。完全可解釋（能指出確切命中詞），但對非預期措辭很脆弱。

方法 **B** — **TF-IDF + 線性 SVM**（`baselines/tfidf_svm.py`）。以字元 n-gram（1-3）TF-IDF 作為特徵 → One-vs-Rest `LinearSVC` 多標籤輸出。它學的是反覆出現的字元模式，而非固定關鍵字。在線上 API 中，SVM 的 `decision_function` 經 sigmoid 轉換成真實的逐議題信心（0.5 的切點正好等於 SVM 原生的判定邊界）。

（BERT 微調列為選擇性的未來工作，刻意不訓練 — 報告與 demo 從不假造 BERT 結果。）

5. 實驗與測試 (Experiments and Testing)

共執行兩項評估。

5.1 保留測試集（229 筆，`eval/metrics.py`）

在固定切分的測試集上計算標準多標籤指標。

5.2 驗收測試套件（1,000 題，`eval/acceptance_suite.py`）

依專案自訂的驗收規格：每類 **50 題 × 20 類 = 1,000 題**，每類再分為 **30 典型 / 10 口語 / 5 模糊 / 5 易混淆 (near-miss)**。near-miss 題刻意安排在易混淆類別旁邊（例如把 過失傷害 寫得接近 傷害；把 不當得利 誤匯 案例寫得接近 詐欺）。各類通過門檻：Top-1 ≥ 0.95 、recall ≥ 0.95 、precision ≥ 0.90 。

誠實提醒：此驗收套件是確定性、模板式的。雖然有 1,000 題，但每類只從 ~3-5 種基礎句型循環，語言多樣性遠低於題數所暗示。它是課程 demo 驗證，不是獨立的人工標註基準。

6. 結果 (Results)

6.1 保留測試集指標

方法	Micro-F1	Macro-F1	Precision	Recall	Hamming Loss
規則比對	0.600	0.426	0.470	0.831	0.062
TF-IDF + SVM	0.760	0.324	0.851	0.686	0.025

6.2 最有意思的發現

我原本以為 TF-IDF + SVM 會在每個指標上都贏。結果並非如此。TF-IDF + SVM 在 Micro-F1 (0.76 vs 0.60)、precision (0.85 vs 0.47)、Hamming loss 都明顯勝出——整體上它是更好、更保守的分類器。但規則比對基準在 **Macro-F1** 反而勝出 (**0.43 vs 0.32**)。

原因是類別不平衡：TF-IDF + SVM 在 **20** 類中有 **9** 類 **F1** 為 **0.000** (最稀有的那些——**偽造文書**、**契約違約**、**肇事逃逸**、**恐嚇** 等)，因為樣本太少根本學不起來，於是它乾脆從不預測這些類別。規則比對仍能靠關鍵字在這些稀有類別觸發，所以即使整體品質較差，它的「逐類別平均」反而較高。這個 Micro 對 Macro 的反轉，是本專案最重要的一課：在不平衡的多標籤任務上，亮眼的整體準確率會掩蓋長尾類別的失敗。

逐類別亮點 (測試 F1)：**酒駕/公共危險** 規則 0.94 / SVM 0.99 (樣本極多)，對比 **恐嚇** 規則 0.56 / SVM 0.00、**肇事逃逸** 規則 0.19 / SVM 0.00。

6.3 驗收測試 (1,000 題)

指標	數值
整體 Top-1 準確率	0.800
整體 Top-3 命中率	0.962
整體 recall@5	0.962
通過嚴格門檻的類別	1 / 20

依情境難度：

難度	題數	Top-1	Top-3	Recall@5
典型 typical	600	0.857	1.000	1.000
口語 colloquial	200	0.735	0.810	0.810
模糊 ambiguous	100	0.730	1.000	1.000
易混淆 near_miss	100	0.600	1.000	1.000

誠實解讀：Top-3 / recall@5 很高 (0.96)——正確議題幾乎總是落在候選集合內。Top-1 在口語與 near-miss 案例下降，這正是關鍵字系統最吃力的地方。20 類只有 1 類通過刻意設嚴的逐類別門檻，

因為該門檻要求稀有類別同時達到 $\geq 95\%$ precision 與 recall——這是目前弱標資料無法達成的標準。此事如實呈報，沒有隱藏。

6.4 解決方案 / 交付系統

- **20** 議題本體論、引文正規化、弱標註與切分流程（可重現）。
- 兩個訓練好的分類器，附誠實比較。
- **Flask API**（`/api/health`、`/api/predict`）回傳標籤、逐議題模型網格，以及法條全文。
- **Next.js** + 設計系統前端（**LexTag**）— 雙語「法律研究儀器」介面：情境輸入、推論動畫、附信心長條的排序預測、雙模型對照（折線圖 + 矩陣，採用真實指標）、可解釋性面板、顯眼的非法律意見免責聲明，以及核心功能——每條支持法條都以可展開卡片顯示實際條文內容，而不只是條號。

實際 API 輸出範例：酒駕情境回傳 `酒駕/公共危險` + 刑法 §185-3 全文；Threads 冒名情境回傳 `妨害名譽` + 刑法 §309、§310 與 民法 §195 全文。

7. 討論 (Discussion)

哪裡做對了。把問題框定為「議題分流」（分類）而非「法律問答」（生成），讓任務可量化、誠實、且真正有用。法條全文查詢把「刑法 §185-3」從不透明的代碼變成可驗證的線索。雙模型對照帶出一個真實、非顯而易見的 ML 教訓（不平衡下的 Micro vs Macro）。

哪裡困難 / 哪裡失敗。規則 + TF-IDF 的組合受限於 (1) 弱標雜訊、(2) 類別不平衡——長尾缺乏資料、(3) 口語化措辭——關鍵字比對抓不到「撞了就跑」（肇事逃逸）或「借了不還」（所有物返還）這類改寫。這些案例會回傳「無預測」並正確觸發「需人工複核」旗標，而非亂猜。

我有兩件事想錯了。第一，我以為 ML 模型會全面壓過規則基準——Macro-F1 反轉證明並非如此。第二，我以為驗收題數越多（1,000）就越嚴謹——但因為它們循環自少數模板，真正的語言覆蓋其實有限，所以報告才明白寫出這點。

未來工作。為稀有類別做人工黃金標註；用 sigmoid 多標籤頭微調 `hfl/chinese-macbert-base` 以捕捉隱含語意；加入分布外（OOD）/「無法律議題」偵測。

8. 結論 (Conclusion)

LexTag 是一條可重現的中文 NLP 流程：以引文推導的弱標把日常法律敘述轉成多標籤議題預測，誠實比較規則與 TF-IDF+SVM 兩種分類器，並以互動 demo 呈現結果，將實際法條內容作為查詢線索。它刻意定位為上游的前置分流 / 導引工具——可量化、可稽核、且不生成法律意見——而這正是它最有用、風險最低的位置。

9. 重現方式與程式碼 (Reproducibility & Code)

```
cd final_project
uv sync --extra dev
uv run python scripts/build_labels.py          # 弱標
uv run python scripts/split_data.py            # 切分 train/val/test
uv run python scripts/build_law_lookup.py       # 法條全文 -> data/law_articles.json
uv run python baselines/rule.py                 # 規則預測
uv run python baselines/tfidf_svm.py           # TF-IDF + SVM
uv run python eval/metrics.py                  # report/results.md
uv run python eval/acceptance_suite.py         # 1,000 題驗收
uv run pytest                                  # 20 項測試
./start_demo.sh                                # Flask API + Next.js demo
```

測試：20 passed。流程與 demo 已端到端驗證。

10. 參考文獻與學術誠信 (References & Academic Integrity)

所有實作均為作者本人完成。所使用的外部資料集、模型與產品引用如下。

資料集

- Huang, Liang-Hsun. [lianghsun/tw-legal-synthetic-qa](#) . Hugging Face.
- Huang, Liang-Hsun. [lianghsun/tw-processed-law-article](#) . Hugging Face.

函式庫

- scikit-learn (TF-IDF、LinearSVC、metrics) ; jieba ; Flask / flask-cors ; PyYAML ; Hugging Face [datasets](#) 。
- Next.js、React、TypeScript (前端) ; Noto Serif/Sans TC + IBM Plex Mono (字體) 。

相關產品 (競品脈絡)

- Lawsnote · Lawbot AI · LawChat · EasyLaw · LawAI (法詢) 。

法條內容取自公開之中華民國法規 (透過 [tw-processed-law-article](#) 資料集) , 僅以查詢線索方式原文呈現。